

- * Mostly used in Banking & Insurance Domain.
- * Java is a Multithreading concept.
- * Thread is a very light weight process in Java, this breaks big problem into small junks (piece), if we run all junks at same time then it is called thread (work happens parallelly). — Multithreading (Multi tasks).
- * Running multiple threads at a time — Multithreading.

ex: package com;

public class Test {

public static void main(String[] args) {

int a = 10/0;

syso(a);

}

}

} This method runs inside main thread. (runs by JVM).

output:

Exception in thread "main" java.lang.ArithmeticException: /

by zero

Syntax to create thread: 2 ways.

1. extends Thread class in Java, after writing a class.
 2. we can implements Runnable (interface).
- run() → override the run method.
- Logic should be write in that run method.

*** Java limits multiple inheritance with classes. so, extends Threads already inherited, due to that we cannot extend another class.

**** But we can implement other class with one class at the same time, so, we use always preferred Thread with implements Runnable interface.

① Extends Thread (Parent of Thread class - Runnable) interface.

Ex: package com;

⇒ public class MyThread extends Thread {

@Override

public void run() {

super.run();

for (int i = 1; i <= 10; i++) {

System.out.println("Thread 1");

String name = Thread.currentThread().getName();

System.out.println("... " + name);

String name = Thread.currentThread().setName("MyThread");

String name = Thread.currentThread().getName();

try { System.out.println("→ " + name);

Thread.sleep(2000);

} catch (Exception e) {

} package com;

public class Test {

public static void main(String[] args) {

Test.sayBye();

Test.sayHi();

MyThread myThread = new MyThread();

myThread.run();

(It gives name of thread) ← String name = Thread.currentThread().getName();

System.out.println(name);

myThread.start(); → [Indication to Thread

Rescheduler] → It will assign new thread to cpu.

[Use instead of myThread.run()] (It will internally call internally)

```

static void sayHi() {
    for (int i = 1; i <= 5; i++) {
        syso("Hi");
        Thread.sleep(2000);
    }
}

static void sayBye() {
    for (int i = 1; i <= 5; i++) {
        syso("Bye");
        Thread.sleep(2000);
    }
}

```

Output:

Thread1	Bye	Hi
Thread1	Bye	Hi
Thread1	Bye	Hi
	Bye	Hi
	Bye	Hi

Thread1.

main. (*Thread name).

----- main.

→ myThread.

Threads Rescheduler:

- * When we create threads, it will collect and assign space to that threads in CPU internally, i.e. registers threads.
- * Without this thread cannot be created.
- * We can register a Thread only with the help of by calling "start()" method.
- * We cannot register a Thread by calling "run()" method.
- * Threads are stored in **stack memory**.
- * Threads are temporary, after completion of process, threads will be erased in stack memory.
- *** * we never expect output in Multi-Threading, due to all threads run parallelly.
(Output will be unpredictable).

package com;

public class MyThread2 extends Thread {

@Override

public void run() {

for (int i=1; i<=10; i++) {

 sysout("Thread 2");

}

 sysout(Thread.currentThread().getName());

try {

3

~~Thread~~ Thread.sleep(2000);

}

^{InterruptedException}
catch (IOException e) {

 sysout(e);

}

Package com;

Public class Text {

Psum {

MyThread2 myThread2 = new MyThread2();

myThread2.start();

myThread2.start();

~~myThread2~~.

text.say Bye();

Output:

(Random outputs)

Bye

Thread1

Thread2

Bye

Thread1

Thread2

MyThread

Output:

main

(Random outputs)

Bye

Bye

Bye

Thread2

Bye

;

Thread1

MyThread

* Class: 29

27/05/2025

Implements Runnable:

Package com;

public class MyThread implements Runnable {

@Override

public void run() {

for (int i = 1; i <= 10; i++) {

System.out.println("Thread");

}

* Create class and implements Runnable.

* If we create an object to class, there is no start() in it.

* Create an object for Thread class & assign to the (custom thread) to that constructor & call with that object.

Ex: Thread t3 = new Thread(myThread);

package com;

public class Test {

public static void main(String[] args) {

Runnable myThread = new MyThread();

MyThread myThread = new MyThread();

myThread.start(); [due to start() is a method of Thread class]
(error)

Thread t3 = new Thread(myThread);

t3.start();

(this accepts Runnable type)

Output: Thread
Thread
Thread

Thread.

}

Ex: package com;

```
public class Counter {
    int count = 0;
    public -synchronized void increment() {
        //count++;
        synchronized(this) {
            count++;
        }
    }
}
```

→ Synchronized block.

package com;

```
public class Thread1 extends Thread {
```

```
    public class MainThread { → Top Level Class.
```

```
        PSVM {
```

[* we can write multiple

```
        Counter counter = new Counter();
```

```
        Thread1 thread t1 = new Thread1(counter);
```

```
        Thread2 t2 = new Thread2(counter);
```

classes in a java but should have only one public class].

```
        t1.start();
```

```
        t1.join();
```

⇒ Output: 1673

```
        t2.start();
```

```
        t2.join();
```

(Random output)

```
        Sysout("Counter value: " + counter.count);
```

Output: 2000 ✓

(Synchronized).

✓ class Thread1 extends Thread {

```
    @Override
```

```
    public void run() {
```

```
        for(int i=1; i<=1000; i++){
```

```
            **
```

```
            MainThread mainThread = new MainThread();
```

```
            mainThread.counter.increment();
```

✓ class Thread2 extends Thread {

```
    @Override
```

```
    public void run() {
```

```
        for(int i=1; i<=1000; i++){
```

```
            MainThread mainThread = new MainThread();
```

```
            mainThread.counter.increment();
```

class Thread1 extends Thread {

Counter counter; (has a relationship)

public Thread1(Counter counter) {

this.counter = counter;

}

@Override

public void run() {

for(int i=1; i<=1000; i++){

//MainThread mainThread = new MainThread();

Counter.increment();

}

}

Output:

Counter value: 0

* join();

current thread

→ This method will stop the execution of (other thread) until the first thread is executed.

main → t1.join();

↓
(hold)

↓
(executed)

* Race Condition: [When Multiple Threads access a resource at a same time, some collision occurs & few iterations are skipped]

* Random Outputs ~~with~~ even executed the taken Threads. got (not getting expected outputs).

* Iteration will happen same time ~~even~~ while the count is not updated. (collision happens).

* If we want to handle this, there is a concept called synchronization.

* Synchronization: (Internally done by JVM) [Thread Safe] → slow

→ It acquires a lock [all threads execution stopped until it executes].

→ Method - synchronized - this method will get lock.

→ Due to this lock process, there is no chance to get collision. (Stops multiple Thread executions)

* Dead Lock in Threads:

* When we stuck some where, if both threads waits for each other, they will go to hold this is called Dead lock.

Ex 1

```
package com;
public class T1 extends Thread {
    T2 t2;
    @Override
    public void run() throws InterruptedException {
        t2.join();
        syso("Thread 1");
        for (int i=1; i<=20; i++){
            syso("From T1");
        }
    }
}
```

```
package com;
public class T2 extends Thread throws InterruptedException {
    T1 t1;
    @Override
    public void run() {
        t1.join();
        syso("Thread 2");
        for (int i=1; i<=20; i++){
            syso("From T2");
        }
    }
}
```

```
package com;
public class Deadlock {
```

```
    Psum {
        t1, t2 = t2; t2.t1 = t1;
        T1 t1 = new T1();
        T2 t2 = new T2();
        t1.start();
        t2.start();
        Syso("Hi");
    }
```

Output: Thread 1
Thread 2

Output: From T2
From T2
...
From T2
From T1
From T1
...
From T1
Hi

No output
but program
is running i.e.
dead lock.
(∵ both are waiting
for each other)

dead lock.

Thread LifeCycle

Stages: 5

* 1. New

2. Runnable

3. Running

4. Blocked / waiting

5. Dead

① New stage:

* Creating new thread & creating an object to that thread.

New Thread T_1 $T_1, t_1 = \text{new } T_1();$ ② Runnable stage:

* In this stage, calls the object, (ready to run, but not running yet).

 $t_1.start();$ ③ Running stage:

Now our thread is actually running by calling $\text{run}()$ method and starts actual execution.

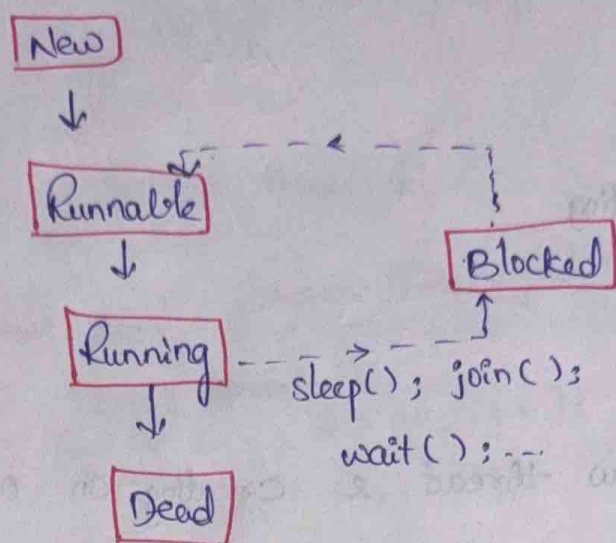
④ Blocked / waiting stage:

* Thread has started and gone blocked { either we called $\text{sleep}()$, $\text{join}()$; }

* $\text{wait}()$; $\text{sleep}()$; $\text{join}()$; using synchronization we can block the execution of threads until current thread executed.

* After $\text{sleep}(1000)$; it will again enter into Runnable stage after that enter into Running stage &

after that enter into dead stage & erase from the stack memory.



wait() & notify() methods: [Coming from ~~Thread~~ Object class].

* These two methods are used in threads.

1. wait():

→ This is called only inside synchronized blocks.

* When wait() is called in main method, main method waits until we call notify() method, after calling the main thread also executes parallelly with other threads.

* Both wait() & notify() works in synchronized block only. They work with only locks.

[IllegalMonitorState exception]

* notify():

* This is called inside synchronized.

* This will inform the calling thread to resume its operation.

* notifyAll():

* When there are multiple threads waiting, then we call notifyAll(); this will inform all waiting threads to resume their operations.

ex: package com;

public class WaitNotify {

public static void main(String[] args) { ^{throws Interrupted Exception}

TestThread t = new TestThread(); ^{→ New}

t.start(); ^{→ Runnable}

[we blocked
main method]

Thread.sleep(1000); ^{→ 100 output}

//syso(t.count); ⁽⁸⁾

t.join(); ^{→ blocked}

output: 0

Random
output.

synchronized(t) {

t.wait(); [main waits until t called notify()]

syso(t.count)

}

//syso(t.count);

output: 100

(after synchronized)

class TestThread extends Thread {

int count = 0;

@Override

public void run() {

*** synchronized(this) {

for (int i = 1; i <= 100; i++) {

count++;

}

this.notify();

}

}

}

* join();

* this will block the current thread until opposite thread completes its execution.

sleep();

* This will block the current thread only for a certain time.

Thread Priority. (1-10)

- * 1 - Least priority
- * 5 - Default priority
- * 10 - highest priority.

* [Priority will not work every time, it depends on space in cpu, memory etc.]

psvm {

syso (Thread.currentThread().getPriority());

output: 5

* Main thread will have a default priority.

* We can modify priority. by,

Thread.currentThread().setPriority(10); output: 10

Thread.currentThread().getPriority();

Ex package com;
public class T10 extends Thread {

@Override

public void run() {

Thread.currentThread().setPriority(10); → output:

syso ("T10");

for (int i=1; i<=5; i++) {

syso ("T10");

Random
output

package com;

public class T15 implements Runnable {

@Override

public void run() {

for (int i=1; i<=5; i++) {

syso ("T15");

}

}

```
package com;
```

```
public class Test {
```

```
    psum {
```

```
        T10 t10 = new T10();
```

```
        T15 t15 = new T15();
```

```
        Thread t = new Thread(t15);
```

```
        t.start();
```

```
        t.start();
```

```
    }
```

```
}
```

output:

Random output.

Ex 6

Exception Handling

```
package com;
```

```
public class Test {
```

```
    public static void main(String[] args) {
```

```
        Test t = new Test();
```

```
        int result = t.calculate();
```

```
        syso(result);
```

output:

Arithmetic Exception
0

[* When we write "return" in finally block, it will override the "return" of catch block.]

```
    }  
    void calculate() {  
        try {  
            int a = 10/0;  
            return a;  
        }
```

```
        catch catch (ArithmeticException e) {  
            syso syso(e);  
            return 0;  
        }
```

```
        finally {  
            syso("From finally");  
            return 1;  
        }
```

output:

Arithmetic Exception
From finally
01

* If method has a return statement, it will exit from the block.